VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Deep Learning and Its Applications (CO3133)

ASSIGNMENT-1

Lecturer	Le Thanh Sach		
Class	CC01		
Team	G-09		
Students	Nguyen Vu Long	2352696	long.nguyenty@hcmut.edu.vn
	Le Phuoc Vu	2353341	vu.lephuocvu13@hcmut.edu.vn
	Nguyen Hoang Quoc	2353027	quoc.nguyenhoang2305@hcmut.edu.vn

Ho Chi Minh, 10/2026

TABLE OF CONTENTS

1 Introduction **2**
1.1 Background and Motivation 2
1.2 Problem Statement 2
1.3 Objectives and Scope 2
1.4 Proposed Approach 2

2 Dataset and Exploratory Analysis **3**
2.1 Dataset Description 3
2.2 Data Preprocessing 3
2.3 Exploratory Data Analysis and Preprocessing 4
2.4 Preprocessing 9

1 Introduction

1.1 Background and Motivation

Image classification máp an image to a category. In an online product catalogue, for example, an deep learning classifier could help sorting images into the correct category. The classification is only one part of the system. The reliability of the result also depends on the dataset, the representation of each image, the separation between training and evaluation data and also the way model quality and computational cost are measured.

Fashion-MNIST provides a practical benchmark for investigating these factors. Its compact grayscale images allow different deep learning architectures and configurations to be evaluated efficiently, while visually similar categories introduce meaningful classification challenges.

This project investigates a range of machine learning and deep learning approaches, from simple baseline models to more advanced architectures. By comparing their predictive performance, generalization ability, training behavior, and computational requirements under a consistent experimental setup, we aim to understand the strengths, limitations, and trade-offs of different approaches to im

1.2 Problem Statement

The task is a supervised, single lable, 10 class image classifications. Let $X \in \{0, \dots, 255\}^{28\times 28}$ be a raw grayscale image and $y \in \{0, \dots, 9\}$ its label. A classifier with parameters θ receives a preprocessed representation $T(X)$ and produces ten real-valued class scores, or logits:

$$f_\theta(T(X)) = \boldsymbol{z} \in \mathbb{R}^{10}, \qquad \widehat{y} = \underset{k\in\{0,\dots,9\}}{\arg\max}\ z_k. \tag{1}$$

The task focuses on assigning a single category to each image. We use this classification problem to compare how different neural network architectures learn and represent image features.

1.3 Objectives and Scope

The objective of this project is to build an end-to-end deep learning pipeline for image classification using the Fashion-MNIST dataset. Through this project, we aim to understand how different neural network architectures process image data and how their designs influence classification performance. We implement and compare five architectures: Linear Classifier, Multilayer Perceptron (MLP), Convolutional Neural Network (CNN), LSTM, and Transformer.

The scope of this project covers the complete workflow, from data exploration and preprocessing to model training and evaluation using PyTorch. All models are trained and evaluated on the same dataset splits to ensure a fair comparison. We analyze their performance using classification accuracy, macro-F1 score, training time, inference time, and model complexity. In addition, we examine training behavior and prediction errors to better understand the strengths and limitations of each architecture.

1.4 Proposed Approach

We use a common experimental pipeline to compare the five architectures. Fashion-MNIST is divided into fixed training, validation, and test subsets, with normalization statistics computed only from the training subset. Each image is represented as a flattened vector for the Linear Classifier and MLP, a two-dimensional grid for the CNN, a sequence of rows for the LSTM, or a sequence of patches for the Transformer. Each model is trained using a shared optimization protocol, and its best checkpoint is selected by the lowest validation loss. The selected models are then evaluated on the test set to compare classification performance, computational cost, and prediction errors.

2 Dataset and Exploratory Analysis

2.1 Dataset Description

Fashion-MNIST contains pieces of fashion images divided into 10 product categories. The dataset included 60,000 train examples and 10,000 of test samples. Each image is a 28 by 28 array unsigned 8-bit gray scale intensities, the measurement range is 0 to 255. Each record has 1 interger label. The table below gives the label mapping used by the source dataset and ours implementation.

Label	**Category**	**Label**	**Category**
0	T-shirt/top	5	Sandal
1	Trouser	6	Shirt
2	Pullover	7	Sneaker
3	Dress	8	Bag
4	Coat	9	Ankle boot

Table 1: Fashion-MNIST Class Labels and Categories

The dataset includes several visually similar categories, such as T-shirt/top and Shirt, or Pullover and Coat, which makes classification more challenging. Despite the small image size, models must learn to distinguish subtle differences in shape and texture to correctly identify each category.

Fashion-MNIST is suitable for this project because its standardized image format allows different neural network architectures to be compared under consistent conditions. However, its low-resolution grayscale images do not fully represent real-world product images, which often contain colors, complex backgrounds, and greater visual variation.

2.2 Data Preprocessing

The official Fashion-MNIST training dataset is divided into training and validation subsets using a stratified 90/10 split with a fixed random seed of 42. This produces 54,000 training images and 6,000 validation images, while the official 10,000-image test set remains separate. Stratification ensures that all ten classes maintain equal representation across the partitions, as shown in Table 2.

Partition	Images	Per class	Role
Training	54,000	5,400	Fit normalization and update model parameters.
Validation	6,000	600	Select the lowest-validation-loss checkpoint.
Official test	10,000	1,000	Evaluate the fixed selected checkpoint.

Table 2: Frozen dataset partitions and their experimental roles. Counts are verified locally.

The training set is used to compute normalization statistics and update model parameters, while the validation set is used to monitor training performance and select the best checkpoint based on validation loss. The test set is reserved for the final evaluation. Figure 1 illustrates the data preparation and evaluation process.

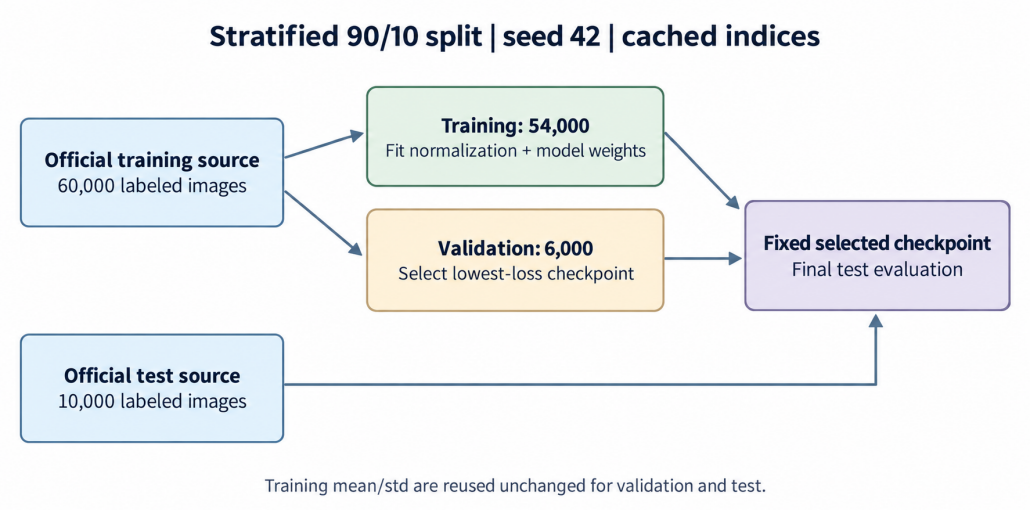


Figure 1: Data Splitting and Evaluation Pipeline

To ensure reproducibility and prevent data leakage, the training and validation indices are fixed and verified to contain no overlapping samples. An additional integrity check is performed using SHA-256 hashes to identify exact duplicate images within and across partitions. As shown in Table 3, no exact duplicates are detected. However, this method cannot identify visually similar images or different images of the same product.

Duplicate Check	Count
Duplicate images within training set	0
Duplicate images within validation set	0
Duplicate images within test set	0
Shared images between training and validation	0
Shared images between training and test	0
Shared images between validation and test	0

Table 3: Duplicate Image Analysis Across Dataset Partitions

2.3 Exploratory Data Analysis and Preprocessing

Exploratory Data Analysis (EDA) is performed on the training dataset to better understand its characteristics before model training. The analysis focuses on class distribution, image variation, pixel intensity, and similarities between categories.

2.3.1 Class Distribution.

The dataset is evenly distributed across all ten categories, with 5,400 training, 600 validation, and 1,000 test images per class, as shown in Figure 3. This balanced distribution helps prevent models from favoring certain classes due to differences in sample counts. Therefore, no additional class balancing techniques are required.

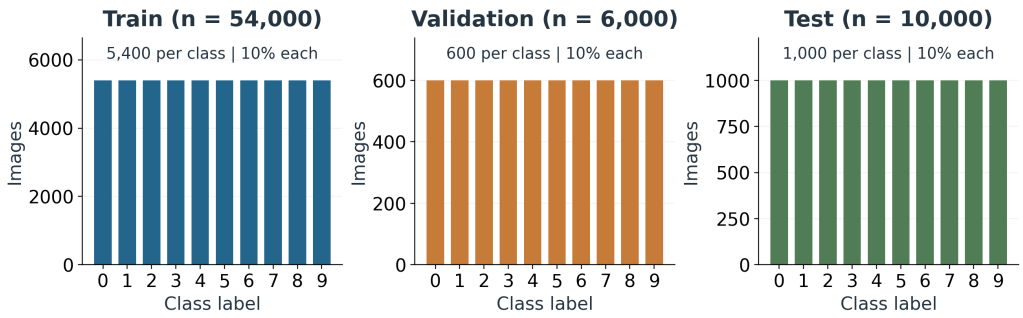


Figure 2: Class Distribution Across Dataset Partitions

2.3.2 Individual Image Variation

Figure 4 presents three randomly selected training images from each of the ten categories using a fixed random seed of 42. The examples highlight variations in shape, brightness, and appearance both within and across categories. Some categories, such as Trouser and Bag, have distinctive shapes that make them visually recognizable. In contrast, upper-body garments such as T-shirt/top, Shirt, Pullover, and Coat share similar features, making them more difficult to distinguish. These observations suggest that classification models need to learn detailed visual features rather than relying only on general shapes to accurately identify each category.

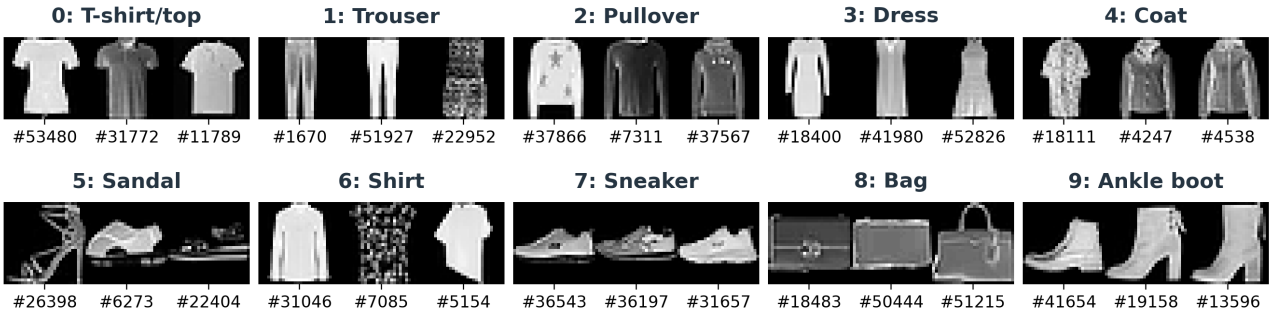


Figure 3: Sample Images from Each Fashion-MNIST Class

2.3.3 Pixel intensity and background

The training dataset contains a total of 42,336,000 pixels, with a mean intensity of 72.98 and a standard deviation of 90.06. Approximately 50.21% of the pixels have an intensity value of zero, indicating that a large portion of the images consists of dark regions. However, none of the training images is entirely blank.

Figure 5 illustrates the pixelintensity distribution. The left panel shows the distribution of all pixel values, highlighting the large proportion of zero-valued pixels. The right panel focuses on nonzero pixels to provide a clearer view of intensity variation.

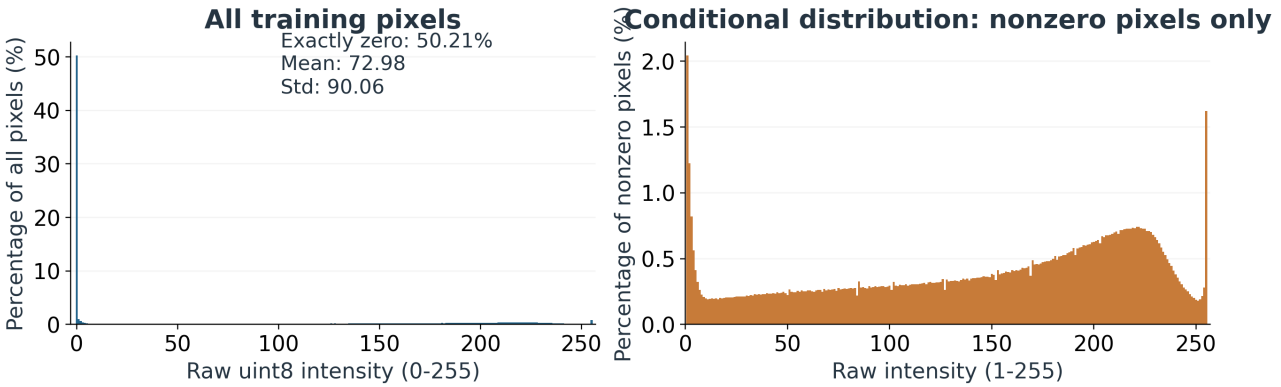


Figure 4: Training Pixel Intensity Distribution

2.3.4 Brightness and Pixel Coverage

Image brightness is calculated as the average intensity of all 784 pixels, ranging from 0 to 255. Nonzero pixel coverage represents the percentage of pixels with intensity greater than zero, providing an approximate measure of how much of an image contains visible information.

Figure 5 illustrates the distributions of brightness and pixel coverage across the ten categories, while Table 4 summarizes their statistics. Coat has the highest mean brightness (98.15), whereas Sandal has the lowest (34.87). In terms of pixel coverage, Pullover has the highest average (64.84%), while Sandal has the lowest (32.10%).

Although these characteristics differ across categories, their distributions overlap considerably. This suggests that brightness and pixel coverage alone are insufficient to distinguish all classes, highlighting the importance of learning more detailed visual features for accurate classification.

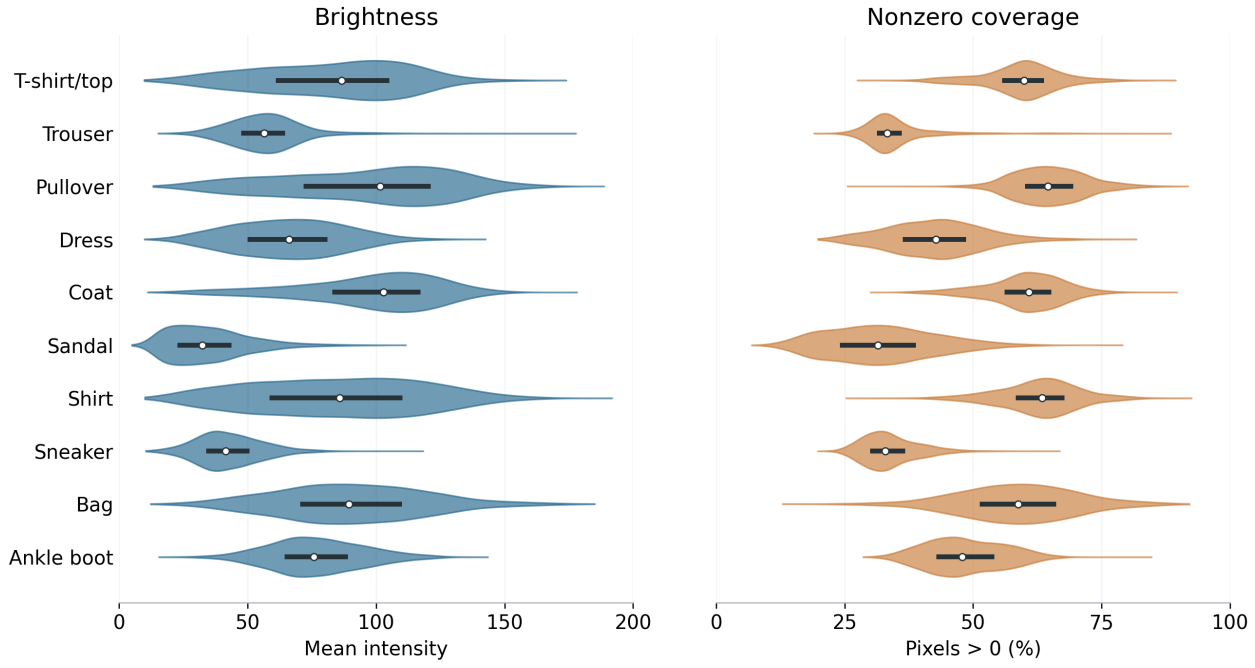


Figure 5: Distribution of Image Brightness and Nonzero Pixel Coverage by Class

Label	Class	Mean Brightness	Brightness SD	Mean Nonzero (%)
0	T-shirt/top	83.02	28.84	59.38
1	Trouser	56.91	14.55	34.91
2	Pullover	96.17	32.70	64.84
3	Dress	65.91	21.33	42.67
4	Coat	98.15	26.46	60.19
5	Sandal	34.87	15.74	32.10
6	Shirt	84.84	32.90	62.87
7	Sneaker	42.81	12.85	33.81
8	Bag	90.28	28.81	58.61
9	Ankle boot	76.87	18.32	48.48

Table 4: Image Brightness and Nonzero Pixel Statistics by Class

2.3.5 Spatial Class Averages and Variability

To examine the spatial characteristics of each class, we compute the mean image and pixel-wise standard deviation from its 5,400 training samples. The mean image represents the typical class appearance, while the standard deviation indicates variations in pixel intensity.

For each class k, these statistics are defined as:

$$\begin{aligned} M_{k,r,c} &= \frac{1}{n_k} \sum_{i \in I_k} U_{i,r,c}, \\ S_{k,r,c} &= \sqrt{\frac{1}{n_k} \sum_{i \in I_k} (U_{i,r,c} - M_{k,r,c})^2}. \end{aligned} \tag{2}$$

where $U_i = X_i/255$ is the normalized image, I_k is the set of samples in class k, $n_k = 5{,}400$, and (r, c) denotes a pixel position.

Figure 6 shows that mean images preserve recognizable class shapes, although some categories, such as pullovers and coats, appear similar. Higher standard deviations occur mainly around object boundaries, reflecting differences in shape and positioning. These patterns highlight the challenges of within-class variation and visual similarity between categories.

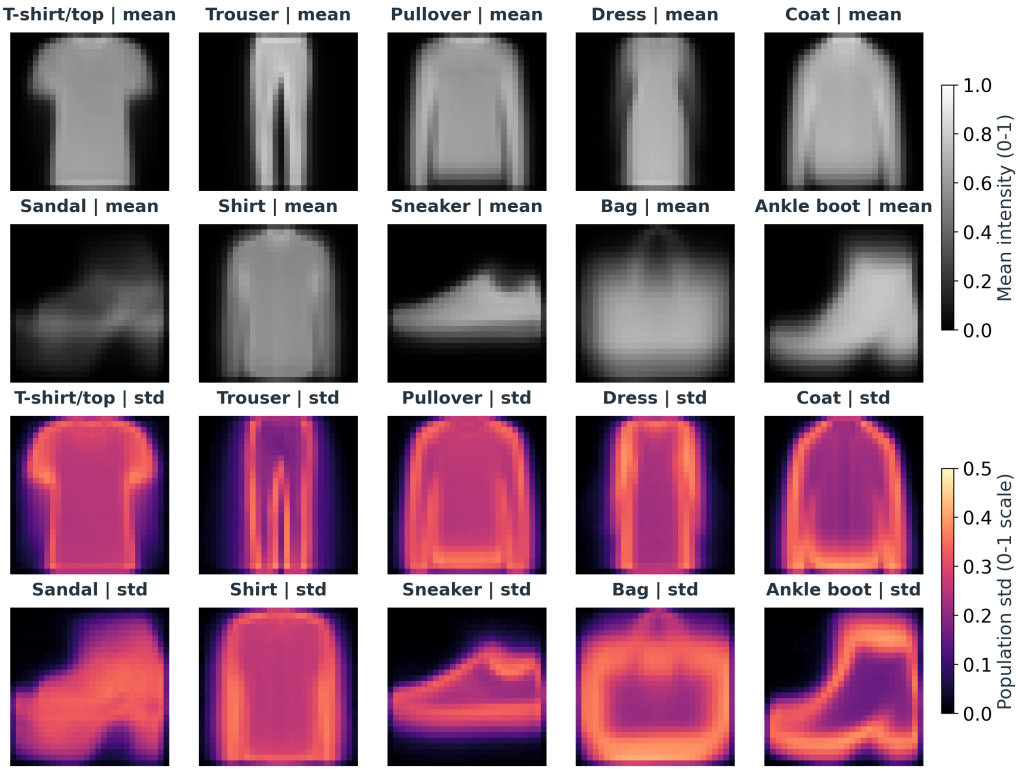


Figure 6: Class-Wise Mean Images and Pixel Variability

2.3.6 Similarity Between Class Prototypes

To quantify similarities between clothing categories, each class mean image is flattened into a 784-dimensional vector. Pairwise cosine similarity is then calculated as:

$$s_{k\ell} = \frac{\mathbf{m}_k^\top \mathbf{m}_\ell}{\|\mathbf{m}_k\|_2 \|\mathbf{m}_\ell\|_2}, \qquad \mathbf{m}_k = \mathrm{vec}(M_k) \in \mathbb{R}^{784}. \tag{3}$$

Figure 7 presents the resulting similarity matrix. The highest similarity between distinct classes is 0.9873, observed between Coat and Shirt, indicating that their average images share highly similar spatial patterns.

These results highlight potential challenges in distinguishing visually similar categories. However, cosine similarity only measures resemblance between class averages and does not account for individual image variations or model predictions. Therefore, the observed similarities will be compared with model confusion matrices during the evaluation stage.

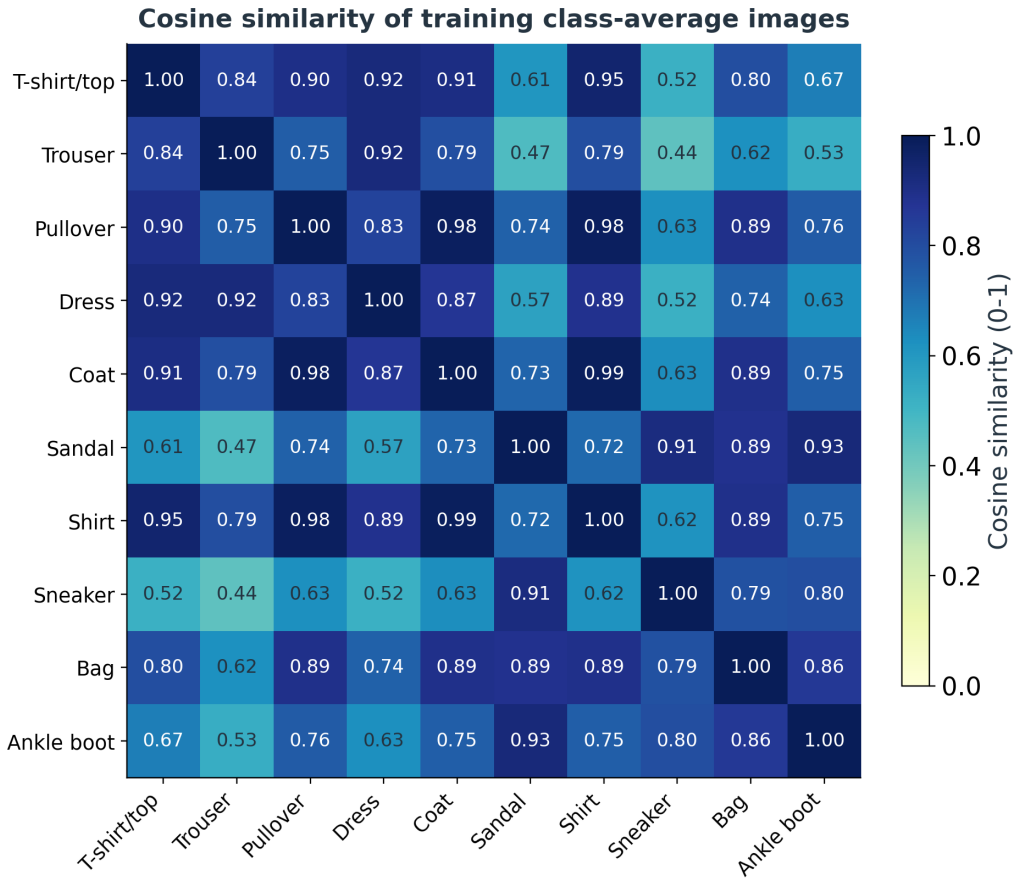


Figure 7: Cosine similarity between training class-average images.

2.4 Preprocessing

After exploring the dataset, a consistent preprocessing pipeline is applied to prepare the images for model training. Although Fashion-MNIST images share the same resolution and grayscale format, their pixel intensities and visual characteristics vary across categories.

The preprocessing process consists of converting images into tensors, scaling and standardizing pixel intensities, and arranging the resulting data into suitable input representations for each neural network architecture. The same numerical transformation is used for all five models to ensure a consistent experimental setup.

2.4.1 Image Conversion and Intensity Scaling

Each Fashion-MNIST image is originally represented as a 28×28 array of unsigned 8-bit integers, with pixel intensities ranging from 0 to 255. Before training, the images are converted into 32-bit floating-point tensors using the `ToTensor` transformation in PyTorch.

This conversion adds a single channel dimension and scales the original pixel intensities to the range $[0, 1]$:

$$U_{i,0,r,c} = \frac{X_{i,r,c}}{255}, \qquad U_i \in [0,1]^{1\times 28\times 28}. \tag{4}$$

$$\begin{aligned} \textit{where} \quad X_i &: \text{ original image,} \\ U_i &: \text{ scaled image,} \\ (r,c) &: \text{ pixel coordinates.} \end{aligned}$$

Scaling places all pixel values within a consistent numerical range, making them suitable for neural network

optimization. The transformation preserves the original image dimensions and relative intensity differences. No resizing or interpolation is required because all images already have the same resolution.

2.4.2 Standardization and Data Leakage Prevention

After scaling, the pixel intensities are standardized using the mean and standard deviation calculated from the training dataset. Unlike intensity scaling, standardization centers the data around the training distribution and expresses pixel values relative to its variability.

The mean μ and population standard deviation σ are calculated across all training pixels:

$$\begin{aligned} \mu &= \frac{1}{N} \sum_{i \in I_{\text{train}}} \sum_{r=0}^{27} \sum_{c=0}^{27} U_{i,0,r,c}, \\ \sigma &= \sqrt{\frac{1}{N} \sum_{i \in I_{\text{train}}} \sum_{r=0}^{27} \sum_{c=0}^{27} (U_{i,0,r,c} - \mu)^2}. \end{aligned} \tag{5}$$

where I_{train} : set of 54,000 training images,
N : total training pixels $(42,336,000)$,
μ : training pixel mean,
σ : training pixel standard deviation.

The calculated statistics are $\mu = 0.286209$ and $\sigma = 0.353160$. Each scaled image is then standardized as follows:

$$Z_{i,0,r,c} = \frac{U_{i,0,r,c} - \mu}{\sigma}. \tag{6}$$

where Z_i denotes the standardized image.

Standardization changes the numerical scale without removing image information. For example, original intensities of 0 and 255 correspond approximately to standardized values of -0.810 and 2.021, respectively. Negative values are expected and are not clipped.

To prevent data leakage, the normalization statistics are calculated exclusively from the training partition. The validation and test datasets use the same precomputed mean and standard deviation without recalculation. This ensures that information from held-out samples does not influence the preprocessing parameters used during training.

2.4.3 Input Representations for Different Models

Although all five architectures receive the same standardized pixel values, their input structures differ according to how each model processes image information. The preprocessing pipeline therefore reshapes the standardized images into the required tensor formats.

Table 5 summarizes the input representations for each architecture.

Model	Input Shape	Representation
Linear / MLP	$(B, 784)$	Flattened pixel vector
CNN	$(B, 1, 28, 28)$	Original image grid
LSTM	$(B, 28, 28)$	28 sequential rows with 28 features
Transformer	$(B, 49, 16)$	49 non-overlapping 4×4 patches

Table 5: Input representations for the five architectures

Here, B represents the batch size, which is set to 128 in the shared training configuration.

2.4.3.1 Linear Classifier and MLP. For the Linear Classifier and Multilayer Perceptron, each standardized image is flattened into a vector containing 784 pixel values. The pixels are arranged in row-major order, preserving their original values and positions within the vector.

This representation allows both models to process the entire image as a single feature vector. However, the input does not explicitly preserve the two-dimensional neighborhood structure used by convolutional operations.

2.4.3.2 Convolutional Neural Network. The CNN receives the standardized image in its original spatial format $(1, 28, 28)$. Preserving the channel, row, and column dimensions allows convolutional filters to learn local visual patterns from neighboring pixels.

No handcrafted features or additional image channels are introduced during preprocessing.

2.4.3.3 LSTM. For the LSTM, each image is represented as a sequence of 28 rows, where each row contains 28 pixel features. The rows are processed in order from the top to the bottom of the image.

This representation allows the recurrent network to examine spatial information sequentially. The sequence describes ordered positions within a static image rather than temporal observations.

2.4.3.4 Transformer. For the Transformer, each image is divided into non-overlapping 4×4 patches. Since the original resolution is 28×28, this produces a 7×7 grid containing 49 patches.

Each patch is flattened into a vector of 16 pixel values, resulting in an input tensor of shape $(49, 16)$ for each image. Patches are ordered from left to right and top to bottom.

The patch extraction can be expressed as:

$$P_{i,7a+b,4u+v} = Z_{i,0,4a+u,4b+v}, \tag{7}$$

$$\begin{aligned} \textit{where} \quad P_i &: \text{ patch representation of image } i, \\ a, b \in \{0, \ldots, 6\} &: \text{ patch coordinates,} \\ u, v \in \{0, \ldots, 3\} &: \text{ pixel coordinates within a patch.} \end{aligned}$$

This produces $49 \times 16 = 784$ values per image, without padding or discarding pixels. The subsequent patch embedding and positional encoding belong to the Transformer architecture rather than the preprocessing stage.

All five representations preserve the same 784 standardized pixel values. Figure 9 illustrates how an identical image is rearranged into the different input formats. Consequently, differences in model performance can be studied under a common input information baseline.

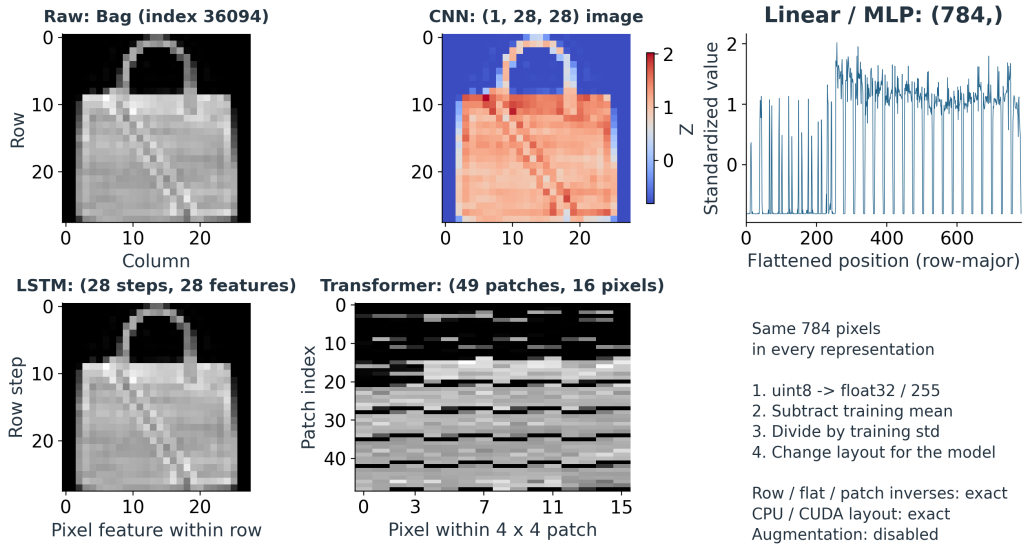


Figure 8: Cosine similarity between training class-average images.

2.4.4 Data Augmentation and Preprocessing Verification

To maintain a consistent baseline, no additional image augmentation is applied during training. Operations such as random rotations, horizontal flips, resizing, denoising, and binarization are excluded from the shared preprocessing configuration.

The EDA also shows that no training image is entirely blank and no exact duplicates are found in the checked dataset partitions. Therefore, no samples are removed based on brightness, pixel coverage, or duplicate detection.

The preprocessing pipeline is verified through numerical and structural consistency checks. The standardized training data has a mean approximately equal to zero and a population standard deviation approximately equal to one, with deviations below 10^{-6}.

Additional checks confirm that flattening, row-sequence conversion, and patch extraction preserve the original pixel values and ordering. Reconstructing an image from its extracted patches also confirms that no pixels are lost during the representation process.

Finally, preprocessing results are checked for consistency between CPU and CUDA operations. These verification steps ensure that all five architectures receive equivalent image information and establish a reproducible foundation for the subsequent training and evaluation stages.